

Salesforce CRM Project Documentation

What Next Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

Project Overview

WhatNext Vision Motors refers to a tailored Salesforce CRM built to support the operational needs of an automotive dealership. The system offers a unified environment where users can manage records related to:

- Vehicles
- Customers
- Dealers
- Vehicle Orders
- Test Drives
- Service Requests

At the core of the application is the order-handling process. Before any order is confirmed, the platform verifies that the vehicle in question is actually in stock, and it is capable of assigning the most suitable dealer to the order automatically, driven by the business rules set up within the org. The build also incorporates automation for sending test-drive reminders and for revisiting orders that remain in a pending state, updating them once the necessary conditions are met.

Through the Lightning interface, users get organised navigation and record layouts, while built-in reports and dashboards support ongoing business review

Objectives

The CRM was built around seven goals:

1. Unify Automotive Records

Bring vehicle, customer, dealer, order, test-drive and service data into one Salesforce environment instead of leaving it spread across disconnected sources.

2. Strengthen Order Management

Move away from unchecked order entry toward a controlled process that confirms vehicle stock before any purchase is allowed to go through.

3. Reduce Repetitive Manual Effort

Offload recurring tasks — matching orders to dealers, sending reminder emails, reconciling pending orders — onto Salesforce Flow and Apex wherever automation made sense.

4. Safeguard Data Quality

Use validation rules and Apex-level checks to stop incomplete or inconsistent transactions from entering the system in the first place.

5. Build an Interface Users Can Work In Easily

Bring everything together inside a Lightning app, with clear navigation, well-organised page layouts and record views, so staff can find what they need without digging around.

6. Give the Business Visibility Into Its Own Activity

Set up reports and dashboards that let the team track order volumes, stock positions and test-drive activity at a glance, rather than relying on manual checks across individual records.

7. Keep Access Under Control

Structure profiles, roles and permission sets so that each user sees and edits only the records relevant to their role, protecting customer and business data as the org grows.

Phase 1 — Requirement Analysis and Planning

Business Requirements

The first step was to look closely at how an automotive dealership actually functions on a daily basis, and where manual handling tends to cause problems. Beyond simply tracking vehicles and customers, the system needed to account for everything that happens around a sale — from a customer browsing the catalogue through to placing an order, booking a test drive, and eventually requesting service. This analysis led to five core requirements:

1. Keep a catalogue of vehicles currently available for sale.
2. Maintain accurate, up-to-date customer records.
3. Track the dealers responsible for fulfilling orders.
4. Record every order placed against a vehicle.
5. Connect each order back to the customer, vehicle and dealer it involves.

Project Scope

- **Vehicle Management** — covers model details, pricing, stock levels and other catalogue information.
- **Dealer Management** — maintains dealership records that support sales and order fulfilment.
- **Customer Management** — stores the buyer information needed across sales, test drives and service interactions.
- **Order and Test-Drive Management** — handles purchase records and scheduled test drives, each tied to the correct customer and vehicle.
- **Service Request Management** — logs post-sale service needs, linked back to the relevant customer and vehicle.

Data Model

The data model captures the key entities that make up the automotive CRM process. A single vehicle can be linked to multiple order and test-drive records, customer details can be tied to the orders and test drives they're associated with, and dealer information connects into the vehicle transactions it supports. Structuring the data this way cuts down on repeated information and makes it much easier to move between related records.

Developer Edition

Search Setup

Setup Home Object Manager

Object Manager

6 items, Sorted by Label

vehicle Schema Builder Create

Label	API Name	Type	Description	Last Modified	Deployed
Vehicle	Vehicle__c	Custom Object		9/11/2026	✓
Vehicle Customer	Vehicle_Customer__c	Custom Object		9/11/2026	✓
Vehicle Dealer	Vehicle_Dealer__c	Custom Object		9/11/2026	✓
Vehicle Order	Vehicle_Order__c	Custom Object		9/11/2026	✓
Vehicle Service Request	Vehicle_Service_Request__c	Custom Object		9/11/2026	✓
Vehicle Test Drive	Vehicle_Test_Drive__c	Custom Object		9/11/2026	✓

Developer Edition

Search Setup

Setup Home Object Manager

Custom Tabs

Help for this Page

Custom Object Tabs

Web Tabs

Visualforce Tabs

Lightning Component Tabs

Custom Object Tabs

Action	Label	Tab Style	Description
Edit Del	Vehicle Customers	Books	
Edit Del	Vehicle Dealers	Building	
Edit Del	Vehicle Orders	Car	
Edit Del	Vehicles	Desk	
Edit Del	Vehicle Service Requests	Cell phone	
Edit Del	Vehicle Test Drives	Car	

No Web Tabs have been defined

No Visualforce Tabs have been defined

Lightning Component Tabs

Security Model

Access control is handled through Salesforce's standard layered model — profiles, roles arranged in a hierarchy, permission sets, and object-level permissions. This gives the org room to grow from a single administrator account during development into a properly segmented, multi-role setup once the CRM is in active use

.

Phase 2 — Backend Development and Configuration

This phase covered building out the actual Salesforce structures — objects, fields, validation and automation — that the requirements from Phase 1 called for. All configuration was carried out directly in Salesforce Lightning Experience.

Custom Objects

Six custom objects were created to model the business, each covering a distinct piece of the customer journey:

Vehicle

Holds catalogue-level data for a car — model, price, current stock count, availability status and the dealer it is tied to. It links outward to Dealer, and is referenced by Order and Test Drive records.

Vehicle Dealer

Represents a single authorised dealership — its name, city or location, and contact channels. It is referenced from Vehicle and pulled into Orders.

Vehicle Customer

Holds a buyer's profile — contact details, address, and the type of vehicle they favour. It is referenced from Orders, Test Drives and Service Requests.

Vehicle Order

Represents one purchase transaction — which customer, which vehicle, when it was placed, and its current status. It looks up to both Vehicle Customer and Vehicle.

Vehicle Test Drive

Captures a booked or completed test-drive appointment, looking up to both Vehicle Customer and Vehicle.

Vehicle Service Request

Logs a post-purchase service or repair request raised by a customer, looking up to both Vehicle Customer and Vehicle.

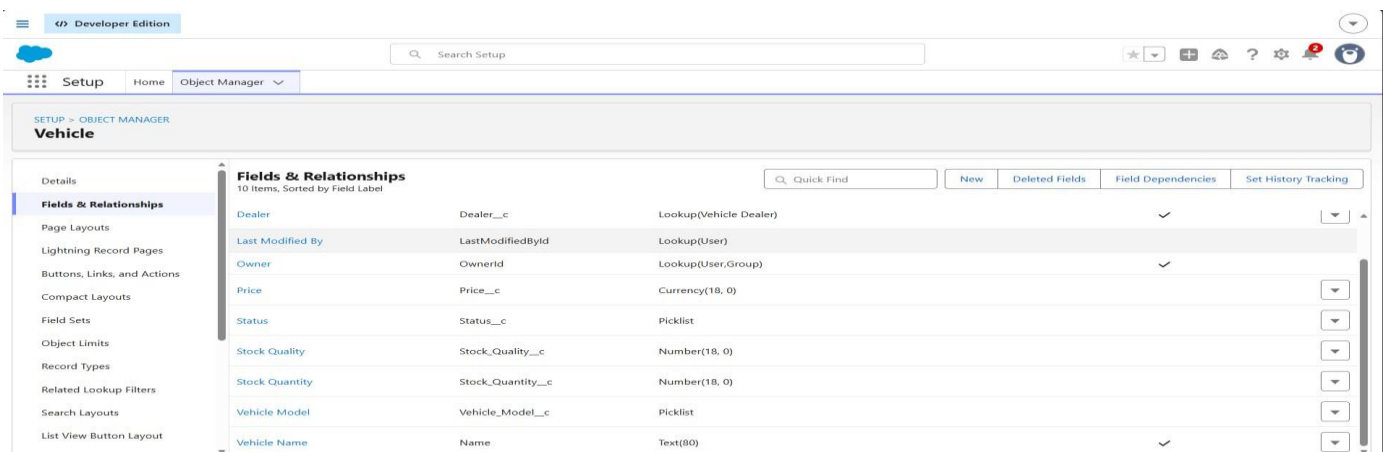
The screenshot shows the Salesforce Setup interface for creating a new custom object. The top navigation bar includes the Salesforce logo, a search bar labeled 'Search Setup', and several utility icons. Below the navigation bar, the 'Setup' menu is open, showing 'Home' and 'Object Manager'. The 'Object Manager' section is active, displaying the 'New Custom Object' page. The page title is 'New Custom Object' with a 'SETUP' icon. Below the title, there are 'Save', 'Save & New', and 'Cancel' buttons. The main section is 'Custom Object Definition Edit'. It contains several input fields: 'Label' (with a red error icon and the text 'Vehicle'), 'Plural Label' (with a red error icon and the text 'Vehicles'), 'Object Name' (with a red error icon and the text 'Vehicle'), and a 'Description' text area. There are also checkboxes for 'Starts with vowel sound' and 'Allow Reports'. A red error message 'Required Information' is visible on the right side of the form. At the bottom, there are links for 'Context-Sensitive Help Setting' and 'Open the standard Salesforce.com Help & Training window'.

The screenshot shows the Salesforce Setup interface for creating a new custom object, specifically the configuration options section. The top navigation bar is the same as the previous screenshot. The 'Setup' menu is open, showing 'Home' and 'Object Manager'. The 'Object Manager' section is active, displaying the 'New Custom Object' page. The page title is 'New Custom Object' with a 'SETUP' icon. Below the title, there are 'Allow Reports', 'Allow Activities', 'Track Field History', 'Allow in Chatter Groups', and 'Enable Licensing' checkboxes. The 'Object Classification' section has a note: 'When these settings are enabled, this object is classified as an Enterprise Application object. When these settings are disabled, this object is classified as a Light Application object. Learn more.' Below this, there are checkboxes for 'Allow Sharing', 'Allow Bulk API Access', and 'Allow Streaming API Access'. The 'Deployment Status' section has radio buttons for 'In Development' and 'Deployed' (selected). The 'Search Status' section has a checkbox for 'Searchable' (selected) and a note: 'When this setting is enabled, your users can find records of this object type when they search. Learn more.' A 'What is this?' link is visible next to the 'Deployment Status' section.

Fields

Each object carries the fields needed to support both day-to-day data entry and the automation layered on top of it:

- **Vehicle** — carries Name, Vehicle Model (picklist), Stock Quantity, Price, a Dealer lookup, and Status (picklist).
- **Vehicle Dealer** — stores Dealer Name, Dealer Location, an auto-numbered Dealer Code, Dealer Phone, and Dealer Email.
- **Vehicle Customer** — captures Vehicle Customer Name, Vehicle Customer Email, Vehicle Customer Phone, Preferred Vehicle Type, and Vehicle Customer Address.
- **Vehicle Order** — includes Vehicle Order Name, a lookup to Vehicle Customer, a lookup to Vehicle, Order Date, and Vehicle Order Status.
- **Vehicle Test Drive** — holds Vehicle Test Drive Name, a Vehicle lookup, a Vehicle Customer lookup, Test Drive Date, and Vehicle Test Drive Status.
- **Vehicle Service Request** — tracks Vehicle Service Request Name, a Vehicle lookup, a Vehicle Customer lookup, Service Date, Issue Description, and Vehicle Service Request Status.



Developer Edition

Search Setup

Setup

Home

Object Manager

SETUP > OBJECT MANAGER

Vehicle Dealer

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

8 Items, Sorted by Field Label

Quick Find

New

Deleted Fields

Field Dependencies

Set History Tracking

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Dealer Code	Dealer_Code__c	Auto Number		
Dealer Email	Dealer_Email__c	Email		
Dealer Location	Dealer_Location__c	Text(18)		
Dealer Name	Name	Text(80)		✓
Dealer Phone	Dealer_Phone__c	Phone		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓

Developer Edition

Search Setup

Setup

Home

Object Manager

SETUP > OBJECT MANAGER

Vehicle Service Request

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

9 Items, Sorted by Field Label

Quick Find

New

Deleted Fields

Field Dependencies

Set History Tracking

Created By	CreatedById	Lookup(User)		
Issue Description	Issue_Description__c	Text(18)		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Service Date	Service_Date__c	Date		
Vehicle	Vehicle__c	Lookup(Vehicle)		✓
Vehicle Customer	Vehicle_Customer__c	Lookup(Vehicle Customer)		✓
Vehicle Service Request Name	Name	Text(80)		✓
Vehicle Service Request Status	Vehicle_Service_Request_Status__c	Picklist		

Developer Edition

Search Setup

Setup

Home

Object Manager

SETUP > OBJECT MANAGER

Vehicle Test Drive

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

8 Items, Sorted by Field Label

Quick Find

New

Deleted Fields

Field Dependencies

Set History Tracking

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Test Drive Date	Test_Drive_Date__c	Date		
Vehicle	Vehicle__c	Lookup(Vehicle)		✓
Vehicle Customer	Vehicle_Customer__c	Lookup(Vehicle Customer)		✓
Vehicle Test Drive Name	Name	Text(80)		✓
Vehicle Test Drive Status	Vehicle_Test_Drive_Status__c	Picklist		

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Preferred Vehicle Type	Preferred_Vehicle_Type__c	Picklist		
Vehicle Customer Address	Vehicle_Customer_Address__c	Text(18)		
Vehicle Customer Email	Vehicle_Customer_Email__c	Email		
Vehicle Customer Phone	Vehicle_Customer_Phone__c	Phone		
Vehicle Customer Name	Name	Text(80)		✓

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Last Modified By	LastModifiedById	Lookup(User)		
Order Date	Order_Date__c	Date		
Owner	OwnerId	Lookup(User,Group)		✓
Vehicle	Vehicle__c	Lookup(Vehicle)		✓
Vehicle Customer	Vehicle_Customer__c	Lookup(Vehicle Customer)		✓
Vehicle Order Name	Name	Text(80)		✓
Vehicle Order Status	Status__c	Picklist		

Validation Rule — Vehicle Stock

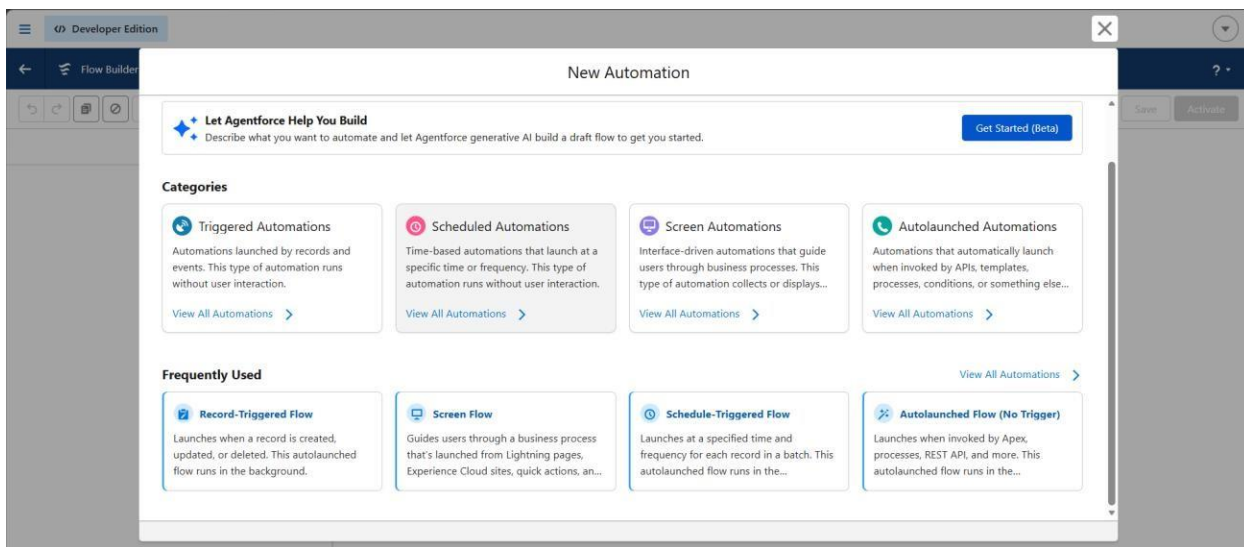
A validation rule guards against confirming an order for a vehicle that has nothing left in stock. If a vehicle's stock quantity is at zero and someone tries to push an order through to Confirmed, the rule intervenes and returns the configured error rather than letting the record save.

Flow Automation

Two parts of the process were shifted onto Salesforce Flow, so they didn't depend on a staff member remembering to step in manually:

- **Dealer Assignment** — a record-triggered flow fires as soon as a new order is created, works out the appropriate dealer for that order, and links the two records together automatically.
- **Test-Drive Reminders** — a flow monitors test-drive records marked as Scheduled and sends the customer a reminder email in the lead-up to their appointment, cutting down on missed bookings.

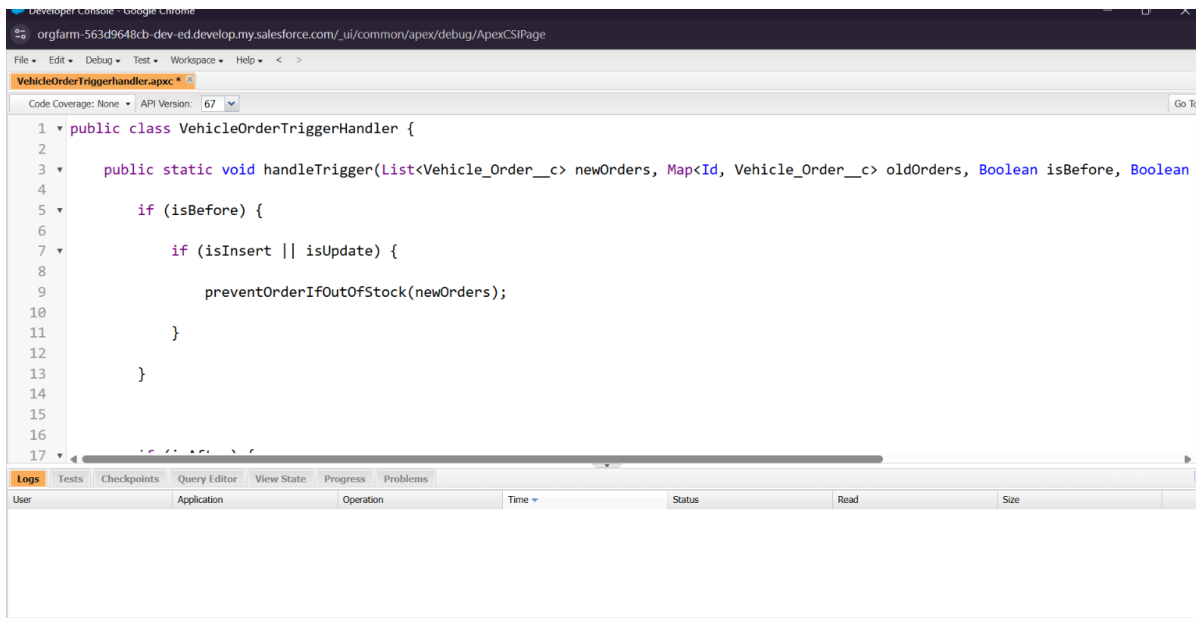
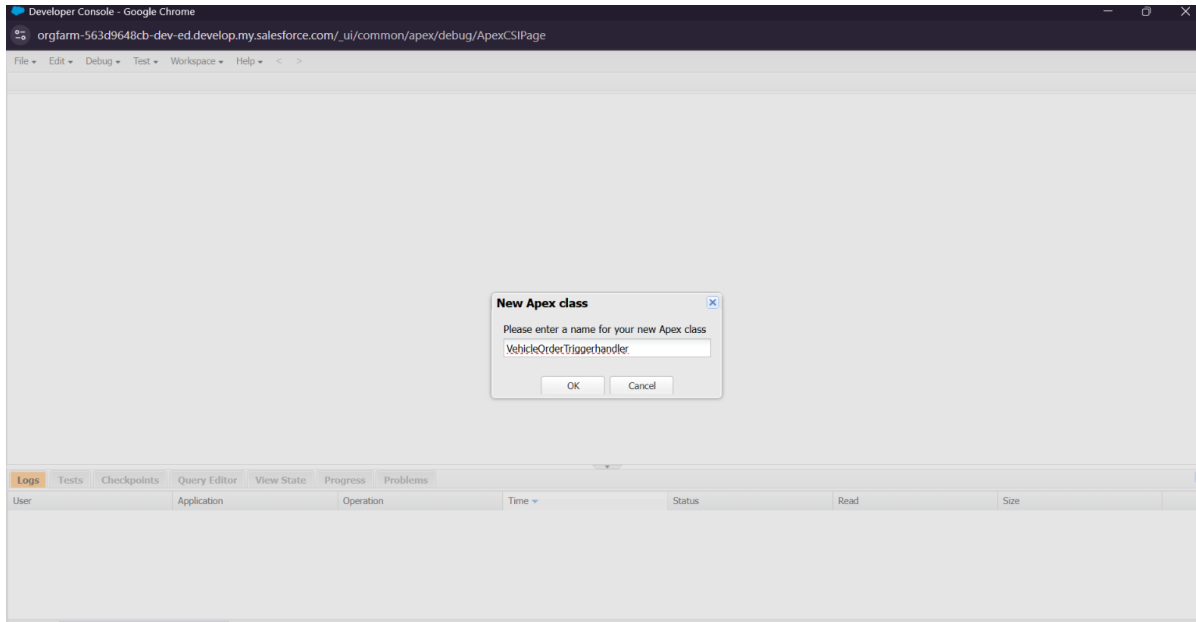
Handling both through Flow rather than manual follow-up keeps the process consistent — every qualifying order gets a dealer, and every scheduled test drive gets a reminder, without relying on someone to catch it each time.



Apex Classes

- **Vehicle Order Processing** — dedicated Apex classes carry the business logic that governs how vehicle orders are handled.
- **Stock Validation** — these classes verify vehicle availability and make sure every order respects the stock conditions defined for it.
- **Order Management** — Apex logic drives the processing and updating of order records as their status changes.
- **Reusable Business Logic** — keeping this logic within separate Apex classes, rather than scattering it inline, makes the implementation simpler to maintain and test over time.

- **Automation Support** — the classes are built to work in tandem with triggers and other Salesforce automation, so required operations run efficiently rather than being duplicated across tools.



Apex Trigger

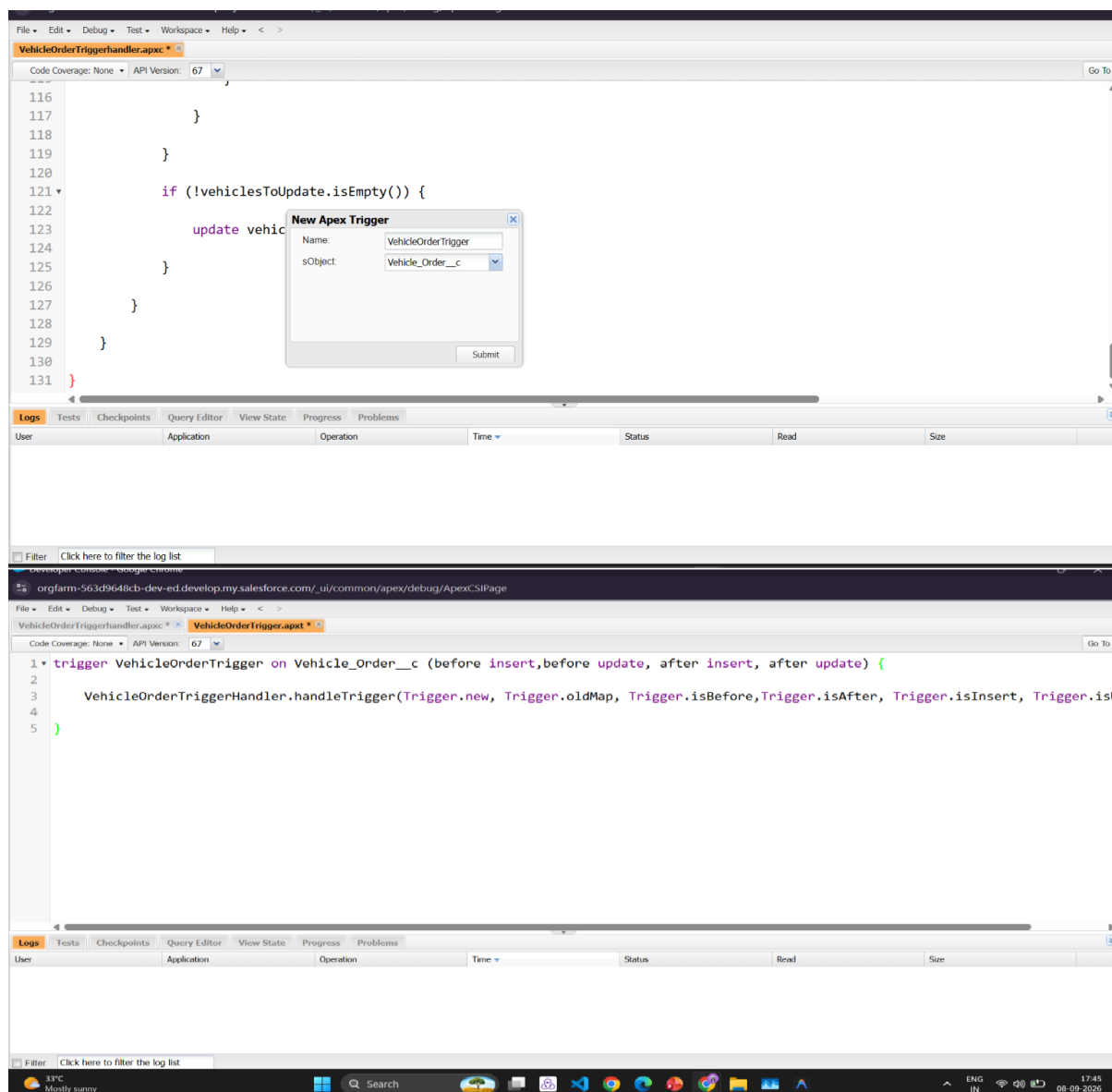
Beyond the Apex classes, the vehicle order process is also supported by trigger-based backend logic. This trigger reacts to relevant events on the

order record and carries out the business checks that have been configured for it.

Its core responsibilities are:

- Verifying that stock is available.
- Stopping an order from processing when conditions aren't met.
- Adjusting vehicle stock once a transaction is confirmed.
- Contributing to the broader handling of order status.

To keep things maintainable, the trigger itself stays lightweight — the actual logic lives in a separate handler class, so the trigger definition and the business rules it enforces remain cleanly separated.



Phase 3 — UI/UX Development and Customisation

This phase was about turning the backend work into something people could actually use comfortably. The scope here covers the Lightning App and App Manager setup, page layouts, Dynamic Forms, user management, reports, dashboards, Lightning Record Pages, and LWC components where relevant.

1. Lightning App Setup via App Manager

The Lightning App provides a centralized interface for managing Vehicles, Dealers, Customers, Orders, Test Drives, and Service Requests. It improves navigation and provides users with easy access to the required Salesforce records and business functions.

The screenshot displays the Salesforce Lightning Experience App Manager interface. The top navigation bar includes 'Setup', 'Home', and 'Object Manager'. A search bar is present with the text 'Search Setup'. The left sidebar shows a list of apps, including 'App Manager', 'External Client Apps', and 'Lightning Out 2.0 Apps'. The main content area shows a list of 31 items, sorted by App Name. The table below lists the first 9 items:

	App Name	Developer Na...	Description	Last Modified Date	App T...	Visib...
1	Agentforce Grid	AIWorkbench	Access Agentforce Grid	8/27/2026, 12:34 AM	Lightning	✓
2	Agentforce Studio	AgentforceStudio	Agentforce Studio	8/27/2026, 12:34 AM	Lightning	✓
3	All Tabs	AllTabSet		8/27/2026, 12:24 AM	Classic	✓
4	Analytics Studio	Insights	Build CRM Analytics dashboards and apps	8/27/2026, 12:24 AM	Classic	✓
5	App Launcher	AppLauncher	App Launcher tabs	8/27/2026, 12:24 AM	Classic	✓
6	Approvals	Approvals	Manage approvals and approval flows	8/27/2026, 12:24 AM	Lightning	✓
7	Automation	FlowsApp	Automate business processes and repetitive tasks.	8/27/2026, 12:33 AM	Lightning	✓
8	Bolt Solutions	LightningBolt	Discover and manage business solutions designed for your industry.	8/27/2026, 12:24 AM	Lightning	✓
9	Community	Community	Salesforce CRM Communities	8/27/2026, 12:24 AM	Classic	✓

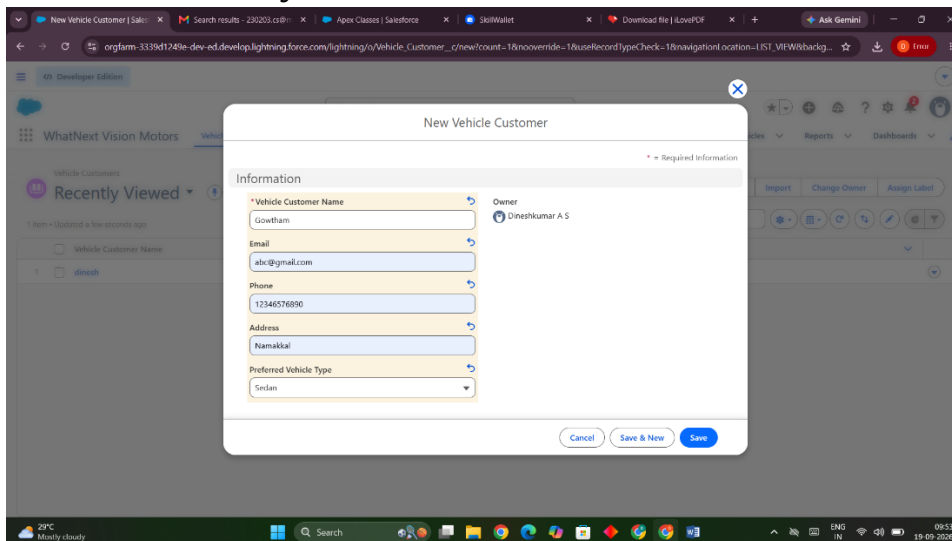
Below the table, a 'New Lightning App' setup form is shown. It includes fields for 'App Name' (WhatNext Vision Motors), 'Developer Name' (WhatNext_Vision_Motors), and 'Description'. There is also a section for 'App Branding' with an 'Image' upload button and a 'Primary Color Hex Value' field (0070D2). The form also has 'Org Theme Options' and a 'Next' button.

2. Page Layouts and Dynamic Forms

Page layouts were set up to control how fields and related information appear on each record, so users see what's relevant without the interface feeling cluttered. Dynamic Forms were applied wherever a field's visibility needed to depend on certain conditions — rather than showing every field at once, the layout adapts based on the values already present on the record.

3. User Management

User accounts were set up in line with the access each person's role actually requires, keeping permissions aligned with real usage rather than granting broad access by default.

A screenshot of a web browser displaying a Salesforce 'New Vehicle Customer' form. The form is a modal window titled 'New Vehicle Customer' with a close button in the top right. It contains a section labeled 'Information' with a list of required fields: 'Vehicle Customer Name' (text input with 'Gowtham'), 'Email' (text input with 'abc@gmail.com'), 'Phone' (text input with '1234567890'), 'Address' (text input with 'Namakkal'), and 'Preferred Vehicle Type' (dropdown menu with 'Sedan' selected). To the right of these fields is an 'Owner' field showing a user profile for 'Dineshkumar A S'. At the bottom of the form are three buttons: 'Cancel', 'Save & New', and 'Save'. The background shows a Salesforce interface with a sidebar and a 'Recently Viewed' list. The browser's address bar shows a Salesforce URL, and the Windows taskbar is visible at the bottom.

4. Reports and Dashboards

Reports were built to give structured, filterable views of the underlying Salesforce data. Dashboards then translate a selection of those reports into a visual summary, letting users get a sense of business activity at a glance instead of working through individual records one by one.

Developer Edition

WhatNext Vision Motors

Vehicle Customers

Vehicle Dealers

Vehicle Orders

Vehicle Service Requests

Vehicle Test Drives

Vehicles

Reports

Dashboards

Vehicle Customer

Gowtham

New Contact

Edit

New Opportunity

Related

Details

Vehicle Customer Name

Gowtham

Email

abc@gmail.com

Phone

(234) 657-6890

Address

Namakkal

Preferred Vehicle Type

Sedan

Created By

Dineshkumar A S · 9/18/2026, 9:24 PM

Owner

Dineshkumar A S

Last Modified By

Dineshkumar A S · 9/18/2026, 9:24 PM

Developer Edition

WhatNext Vision Motors

Vehicle Customers

Vehicle Orders

Vehicle Dealers

Vehicle Service Requests

Vehicle Test Drives

Vehicles

Reports

Dashboards

Vehicle Order

O-0004

New Contact

Edit

New Opportunity

Related

Details

Vehicle Order Number

O-0004

Vehicle Customer

Gowtham

Vehicle

Honda

Order Date

9/22/2026

Status

Confirmed

Assigned Dealer

TM

Created By

Dineshkumar A S · 9/18/2026, 9:27 PM

Owner

Dineshkumar A S

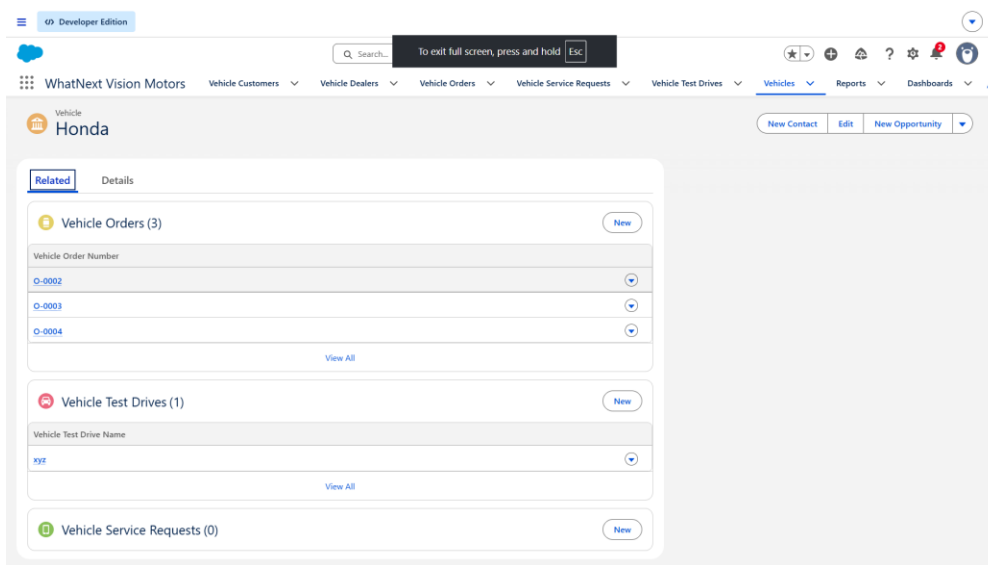
Last Modified By

Dineshkumar A S · 9/18/2026, 9:27 PM

5. Lightning Record Pages

Lightning Record Pages were configured so that each record presents useful, well-organized information. Relevant components typically include:

- Record details
- A highlights panel
- Related lists
- Tabs



Phase 4 — Data Migration, Testing and Security

This phase covered getting real data into the CRM, confirming that everything built so far actually worked, and putting the right security controls in place. It touches on data loading, history tracking, duplicate and matching rules, profiles, roles, permission sets, sharing rules, and testing.

1. Data Loading

Sample data was organised in a structured format before being brought into Salesforce. The process followed a consistent sequence:

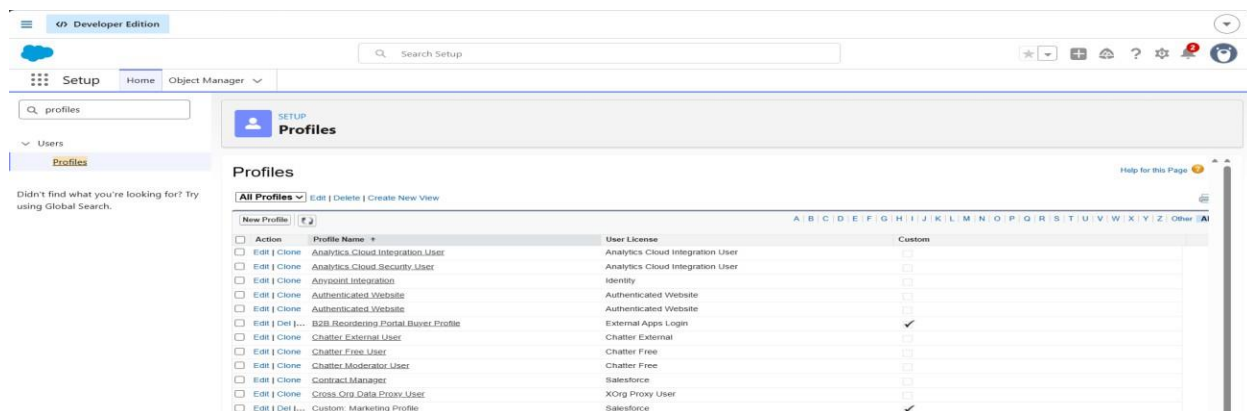
1. Drafting sample records.

2. Matching each data field to its corresponding Salesforce field.
3. Loading the records into the org.
4. Confirming the records were created accurately.
5. Checking that relationships between related records had come through correctly.

Depending on the object involved and the volume of records, either the Data Import Wizard or Data Loader would be the appropriate tool for larger-scale loading.

2. Profiles

Profiles form the baseline permission layer for every Salesforce user, setting the boundaries for what applications, objects, fields, and other functionality a person can access.

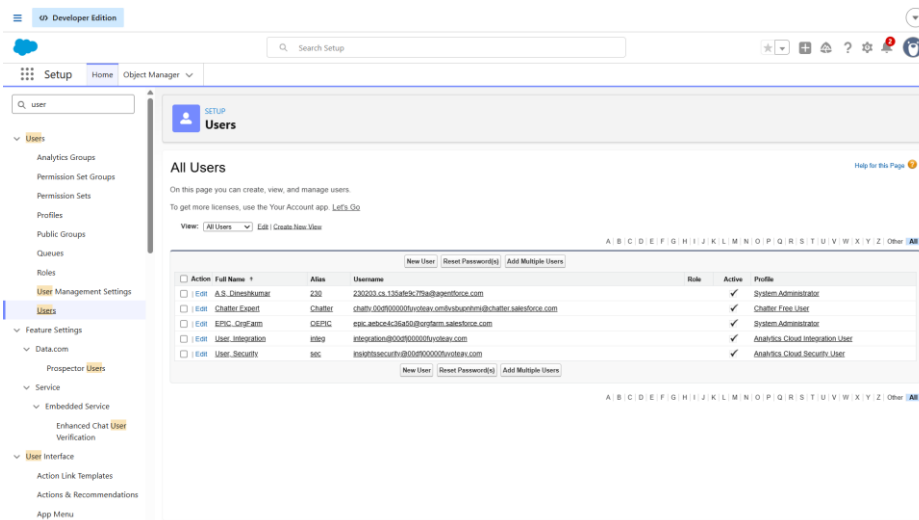


3. Roles and Role Hierarchy

The role hierarchy reflects the organisation's reporting structure and feeds into how record access is determined. One possible arrangement for this project runs:

CEO → Sales Manager → Sales Representative

The hierarchy actually configured within the org should be captured in the final screenshots, since the structure above is illustrative rather than fixed.



4. Permission Sets

Permission sets were used to extend access for users who need specific functionality beyond what their base profile provides.

5. Sharing Rules

Sharing settings govern how records are exposed across different users. The sharing model was designed to match each role's actual responsibilities — giving users the access they need to do their job, while keeping records outside their remit appropriately restricted.

Test cases were prepared for each major Salesforce feature — record creation, approval processes, automatic task generation, flows, and triggers — with input data and resulting output captured as supporting screenshots

Test Case 1 — Test-Drive Reminder Email

Confirms that the automated reminder process fires correctly ahead of an upcoming test drive.

Test Data:

- Customer: An existing customer record
- Customer Email: A valid email address
- Status: Scheduled
- Test Drive Date: The following day

Developer Edition

Flow Builder

Run Immediately

Get Customer Information.
Get Records

Send Test Drive Reminder
Action

End

Send Email

*Label
Send Test Drive Reminder

*API Name
Send_Test_Drive_Reminder

Description

Use values from earlier in the flow to set the inputs for the "Send Email" core action. To use its outputs later in the flow, store them in variables.

Set Input Values

Configure Recipient Details

Recipient Addresses

Developer Edition

Flow Builder

Auto Assign Dealer - V1

Last saved on 9/5/2026, 08:02 PM Active Run Debug View Tests Save As New Version

Record-Triggered Flow
Start

Object: Vehicle Order
Trigger: A record is created
Optimize for: Actions and Related Recor...
+ Add Scheduled Paths (Optional)
Open Flow Trigger Explorer for Vehicle...

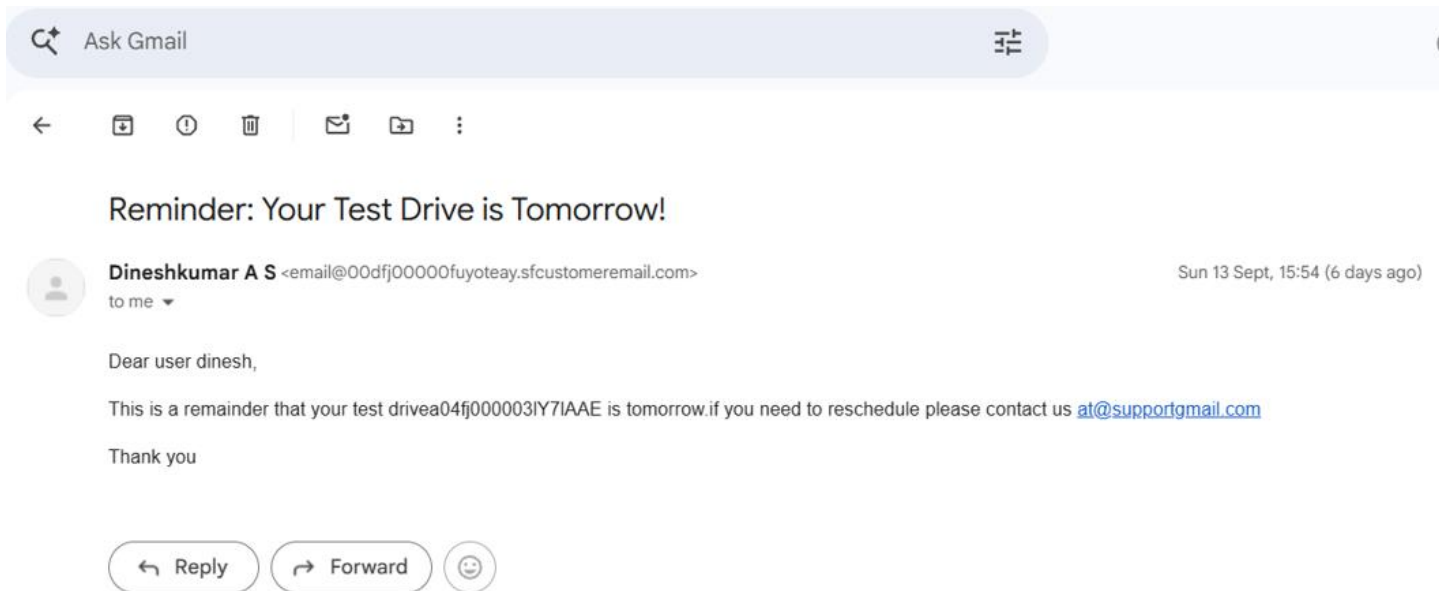
Run Immediately

Get Customer Information
Get Records

Get Nearest Dealer
Get Records

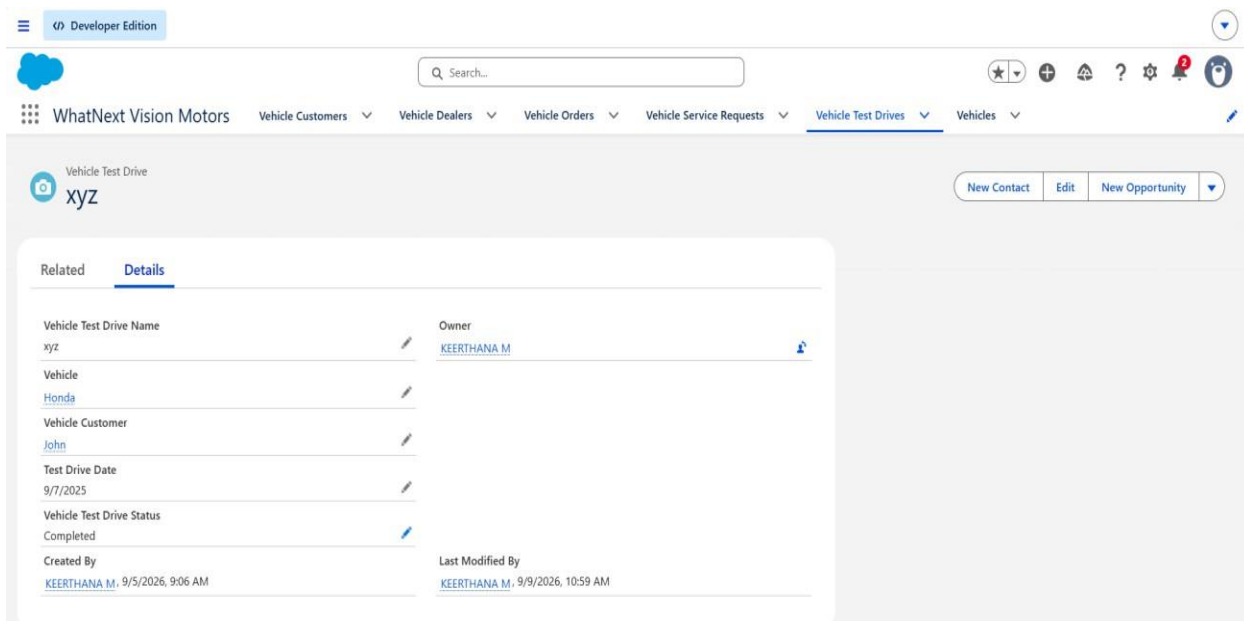
Assign Dealer to Order
Update Records

End



✓ Test Case 4 — Pending Order Batch Processing

Confirms that an order left in a pending state is correctly processed once the required stock becomes available.



Phase 5 — Deployment, Documentation and Maintenance

Deployment Strategy

Salesforce's supported metadata-transfer mechanisms carry the build from development into whichever environment it needs to reach next. For this project, that means moving the custom objects, fields and relationships, validation rules, flows, Apex classes and triggers, Lightning pages and reports together as one coherent package, so nothing gets left behind and dependencies between components stay intact.

Maintenance and Monitoring

A few areas of Setup give ongoing visibility into how the automation is performing:

- Apex Jobs — for tracking batch and scheduled Apex runs.
- Debug Logs — for digging into unexpected Apex or automation behaviour.
- Flow errors — surfaced automatically when a flow run fails, pointing toward where the problem lies.
- User access reviews — periodic checks to confirm users still hold only the permissions they actually need.

Troubleshooting Approach

Issues are worked through in a consistent sequence rather than guesswork:

1. Pin down which feature is producing the unexpected result.
2. Reproduce it with controlled test data.
3. Review the relevant configuration — object, fields, validation rule, flow, permissions or Apex logic.
4. Check debug logs where programmatic logic is involved.
5. Correct the underlying configuration or code.
6. Re-run the original scenario to confirm the fix holds.
7. Deploy the correction through the standard deployment process once testing passes.

Conclusion

The WhatNext Vision Motors build gives the business a working Salesforce environment that brings its core automotive processes onto one platform. Vehicles, customers, dealers, orders, test drives and service requests are all connected through a deliberate data model rather than living as isolated records. Stock is checked before an order is allowed to proceed, dealer assignment happens automatically, and test-drive reminders and pending-order reconciliation run without staff needing to chase them manually.

Beyond the automation itself, the project reflects a Lightning interface built for actual daily use, a security model laid out around profiles, roles and permission sets, and a testing process that leaves behind concrete evidence of what works. Taken together, it demonstrates how Salesforce's declarative tools and Apex development can be combined to solve a genuine business problem rather than treated as separate exercises.

